

Day 28



Javascript
THE LANGUAGE OF THE WEB

Callback Hell & Pyramid of Doom:

What is Callback Hell?

Callback Hell happens when you have too many callbacks inside each other, making the code look messy, confusing, and hard to read.

Simply:

Callback Hell = too many nested functions.

What Causes Callback Hell?

Callback hell happens when:

- Each task depends on the previous one
- And each task uses a callback
- So you keep nesting functions inside functions

Example:

Task 1 → inside it Task 2 → inside it Task 3 → inside it Task 4

This creates deep nesting.

What is the Pyramid of Doom?

When callbacks keep going inward (right side more and more), the code shape looks like a pyramid or stairs.

Example: callback hell

```
JS callback_hell.js > setTimeout() callback
1  setTimeout(() => {
2    console.log("Step 1");
3    setTimeout(() => {
4      console.log("Step 2");
5      setTimeout(() => {
6        console.log("Step 3");
7        setTimeout(() => {
8          console.log("Step 4");
9        }, 1000);
10     }, 1000);
11   }, 1000);
12 }, 1000);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day21-30\Day28> node .\callback_hell.js
Step 1
Step 2
Step 3
Step 4
```

Why is Callback Hell Bad?

- Hard to read
- Hard to understand
- Hard to debug
- Hard to maintain
- Hard to add new code
- Errors become difficult to handle

Solution: Use **Promises** or **async/await**.

--The End--